S2S implementation

This document explains how to implement an **S2S (server-to-server) event flow** for a mobile game that uses Firebase Authentication, Firebase App Check, Adjust attribution and an Adjust S2S API. The solution consists of a Unity client (C#) and a Node.js Firebase Cloud Function. Each section expands on the high-level steps and includes code examples, diagrams and references to official documentation.

Client-side process

1. **App verification with Firebase App Check** – App Check protects the backend from traffic that doesn't originate from your genuine app. In a Firebase Cloud Function, the `X-Firebase-AppCheck` header is sent to the server; the Admin SDK verifies it and the request is rejected if the token is invalid. In Unity you enable App Check in the Firebase console and set `SetTokenAutoRefreshEnabled(true)` to ensure the token is refreshed automatically. When calling your cloud function you must include this token in the `X-Firebase-AppCheck` header (the Firebase Functions SDK in Unity does this automatically).

   ```
   // Initialise Firebase and App Check (C#)
   using Firebase.AppCheck;
   FirebaseAppCheck.SetAppCheckProviderFactory(new PlayIntegrityProviderFa
   ctory());
   FirebaseAppCheck.SetTokenAutoRefreshEnabled(true);
   // later when calling the function the SDK includes the App Check token
   automatically
   ```

2. **User authentication** – Your client must authenticate users with either Firebase Authentication (e.g., Google, email/password) or a custom authentication provider. A signed-in user can obtain an ID token via `FirebaseAuth.DefaultInstance.CurrentUser.GetIdTokenAsync()`; this token is sent with requests to the cloud function. On the server the Admin SDK verifies this token using `verifyIdToken()`

   ```
   // Sign in anonymously (or with a provider)
   var auth = FirebaseAuth.DefaultInstance;
   await auth.SignInAnonymouslyAsync();
   var user = auth.CurrentUser;
   string idToken = await user.TokenAsync(true); // fetches a fresh ID tok
   en
   // Pass idToken in Authorization header when making HTTP request
   ```

3. **Call server event only if authenticated** – Before sending a server-to-server event, verify that `auth.CurrentUser` is not null and that you have successfully obtained an ID token. Avoid calling the cloud function from unauthenticated clients.

4. **Obtain attribution information using Adjust attribution callback** – The Adjust Unity SDK provides a delegate that fires when the SDK receives new attribution data. Register the delegate before initialising the SDK; when triggered, inspect the campaign name, ad group and creative fields to determine whether the user came from an offerwall promotion

```
using com.adjust.sdk;

public class AdjustAttributionHandler : MonoBehaviour
{
    void Awake()
    {
        Adjust.attributionChanged += OnAttributionChanged;
        AdjustConfig config = new AdjustConfig("YOUR_APP_TOKEN", Adjust
Environment.Production);
        Adjust.start(config);
    }

    private void OnAttributionChanged(AdjustAttribution attribution)
    {
        // filter campaigns that match your offerwall names
        if (attribution.campaign == "offerwall_campaign")
        {
            // store relevant identifiers or flags for later
        }
    }
}
```

5. **Filter campaigns matching offerwall promotion names** – From the `AttributionAttribution` object in the callback, check the `campaign`, `adgroup` and `creative` properties. Only allow S2S events for campaigns that match your configured offerwall promotion names. This prevents awarding rewards for unrelated campaigns.

6. **Call S2S event via Firebase Functions** – When the user completes a rewardable action (e.g., finishing a level), build a JSON payload that includes the event type, metadata (such as the level number) and unique device identifiers required by Adjust. Call the HTTPS Cloud Function using the Firebase Functions SDK or `UnityWebRequest`, including the ID token in the `Authorization` header and App Check token in `X-Firebase-AppCheck` header.

```
using UnityEngine.Networking;
using System.Collections;

IEnumerator ReportLevelComplete(int level)
{
    var user = FirebaseAuth.DefaultInstance.CurrentUser;
    string idToken = await user.TokenAsync(true);
    string functionUrl = "https://REGION-PROJECT.cloudfunctions.net/rep
```

```
ortEvent";
    UnityWebRequest request = new UnityWebRequest(functionUrl, "POST");
    string jsonBody = JsonUtility.ToJson(new
    {
        eventType = "level_complete",
        levelNumber = level,
        deviceId = SystemInfo.deviceUniqueIdentifier
    });
    byte[] bodyRaw = System.Text.Encoding.UTF8.GetBytes(jsonBody);
    request.uploadHandler = new UploadHandlerRaw(bodyRaw);
    request.downloadHandler = new DownloadHandlerBuffer();
    request.SetRequestHeader("Content-Type", "application/json");
    request.SetRequestHeader("Authorization", "Bearer " + idToken);
    // App Check token is automatically added when using the Firebase F
unctions SDK.  With UnityWebRequest you would add it manually.
    yield return request.SendWebRequest();
    if (request.result == UnityWebRequest.Result.Success)
    {
        Debug.Log("Event reported: " + request.downloadHandler.text);
    }
    else
    {
        Debug.LogError("Error reporting event: " + request.error);
    }
}
```

Server-side process (Firebase Cloud Functions)

The backend uses Node.js Firebase Cloud Functions. You must enable the Blaze plan to deploy HTTP functions.

1. **Verify the user and App Check tokens** – When the HTTP function receives a request it must verify the ID token and App Check token. The Admin SDK provides `getAuth().verifyIdToken(idToken)` to decode the ID token. App Check tokens are available in the `X-Firebase-AppCheck` header and are verified with `getAppCheck().verifyToken()`. If either verification fails, return HTTP 401.

```
const functions = require('firebase-functions');
const admin = require('firebase-admin');
const axios = require('axios');
admin.initializeApp();

exports.reportEvent = functions.https.onRequest(async (req, res) => {
  // Verify App Check
  const appCheckToken = req.headers['x-firebase-appcheck'];
  try {
    await admin.appCheck().verifyToken(appCheckToken);
  } catch (err) {
    return res.status(401).send('AppCheck verification failed');
  }
```

```
  // Verify ID token
  const idToken = (req.headers.authorization || '').split('Bearer ')[1]
;
  let decoded;
  try {
    decoded = await admin.auth().verifyIdToken(idToken);
  } catch (err) {
    return res.status(401).send('Invalid ID token');
  }

  const uid = decoded.uid;
  const body = req.body;
  // Validate event
  if (body.eventType !== 'level_complete') {
    return res.status(400).send('Unsupported event type');
  }
  const levelNumber = body.levelNumber;
  const userDoc = await admin.firestore().collection('users').doc(uid).
get();
  const lastLevel = userDoc.get('lastLevel') || 0;
  if (levelNumber <= lastLevel) {
    return res.status(400).send('Invalid level number');
  }

  // Send S2S event to Adjust
  const adjustUrl = 'https://s2s.adjust.com/event';
  const params = {
    app_token: process.env.ADJUST_APP_TOKEN,
    event_token: process.env.ADJUST_EVENT_TOKEN,
    s2s: 1,
    environment: 'production',
    gps_adid: body.deviceId,
    callback_params: JSON.stringify({ uid: uid, level: levelNumber }),
  };
  try {
    await axios.get(adjustUrl, { params, headers: { Authorization: `Bea
rer ${process.env.ADJUST_AUTH_TOKEN}` } });
  } catch (err) {
    console.error('Adjust S2S error', err.response && err.response.data
);
  }

  // Update database
  await admin.firestore().collection('users').doc(uid).update({ lastLev
el: levelNumber });
  return res.status(200).json({ status: 'ok' });
});
```

2. **Validate the event** – The function checks that the `eventType` is supported and that the level number is greater than the last completed level stored in Firestore. This protects against fraudulent calls where the client might attempt to report unrealistic progress.

3. **Fire Adjust S2S event** – Use `axios.get()` or `axios.post()` to call the Adjust S2S API. You must pass `app_token`, the event-specific `event_token` and `s2s=1` along with at least one device identifier (e.g., `gps_adid` for Android). Adjust's documentation stresses that S2S events must include these parameters and should be sent chronologically and within 58 days of occurrence. Authentication is handled by sending an `Authorization: Bearer` header with your API token. You can attach callback parameters such as user ID or level to receive them back in Adjust callbacks.

4. **Update the user database** – After the event is successfully sent to Adjust, update the user's progress in Firestore or the Realtime Database so that subsequent events can be validated correctly.

Adjust side process

Adjust's dashboard and APIs are used to configure tracking and test S2S events.

1. **Create two sets of events** – Define one set of events for client-side tracking (e.g., installs, session starts) and another set for S2S reward events. Each event receives a unique **event token** from the Adjust dashboard. Use distinct tokens to distinguish client and server events.

2. **Use the testing console** – Adjust provides a testing console that displays incoming events in real time. After implementing the S2S call, fire test events from your development environment and verify that the events appear with the correct parameters.

3. **Generate tokens** – An **app token** uniquely identifies your app in Adjust. **Event tokens** are created per event when configuring your campaign. For S2S API calls you also need an **authentication token**; this token is available in your Adjust user profile and must be provided as a Bearer token in the `Authorization` header.

4. **Required parameters** – When sending S2S events you must include `app_token`, `event_token` and set `s2s=1`. At least one device identifier (`adid`, `gps_adid`, `idfa` or other) must be provided; a timestamp can be included as `created_at` if the event occurred earlier than the request time. Adjust expects events to be submitted in the order they occurred and no later than 58 days after the event.

Server structure and requirements

1. **Setup prerequisites** – Install Node.js LTS (version 14 or 16), then install the Firebase CLI globally:

```
npm install -g firebase-tools
```

Authenticate with your Google account using `firebase login` and create or select a Firebase project. Initialise Cloud Functions by running `firebase init functions`. The CLI scaffolds a project with a `functions` directory, `package.json`, `.firebaserc` and `firebase.json`. Set the runtime to Node.js 14 or 16 when prompted.

2. **Project folder structure** – After initialisation the project contains:
   - **functions/index.js** – entry point for your cloud functions.
   - **functions/package.json** – declares dependencies like `firebase-admin` and `firebase-functions`.
   - **firebase.json** – configuration for Cloud Functions and other Firebase services.
   - **.firebaserc** – stores your project ID.
3. **Testing on local emulators** – The Firebase CLI includes an emulator for Cloud Functions. Run `firebase emulators:start` to spin up the emulator and test your functions locally. The emulator serves HTTP functions at a URL like `http://localhost:5001/<project-id>/us-central1/reportEvent`【829260430558549†L1387-L1474】. Use curl or your Unity client to send requests. Ensure you have installed compatible versions of the `firebase-admin` (≥ 8.0) and `firebase-functions` (≥ 3.0) packages.
4. **Deployment** – Deploy your functions to production using the CLI: `firebase deploy --only functions`. After deployment the Firebase console displays the function URL for each HTTP function. Deploying Node.js 14 or 16 functions requires the Blaze plan.

Server request handling process

This section demonstrates how requests are made from Unity and how they are validated on the server.

1. **Unity client call** – The earlier `ReportLevelComplete` coroutine shows how to call your function using `UnityWebRequest`. You must include the ID token in the `Authorization` header and optionally the App Check token in `X-Firebase-AppCheck` (when not using the Firebase Functions SDK). Pass the event type, event parameters and device identifiers in the request body.
2. **Parsing and validating the request** – On the server, parse the request body (e.g., using Express or the raw Node.js body). Extract the ID token from the `Authorization` header and call `admin.auth().verifyIdToken()` to validate it. Extract the App Check token from `X-Firebase-AppCheck` and verify with `admin.appCheck().verifyToken()`. Check that the event type is expected and that the parameters (like level number) are consistent with data stored in Firestore. Reject any mismatch with a 400 error.

3. **Valid vs invalid calls –**

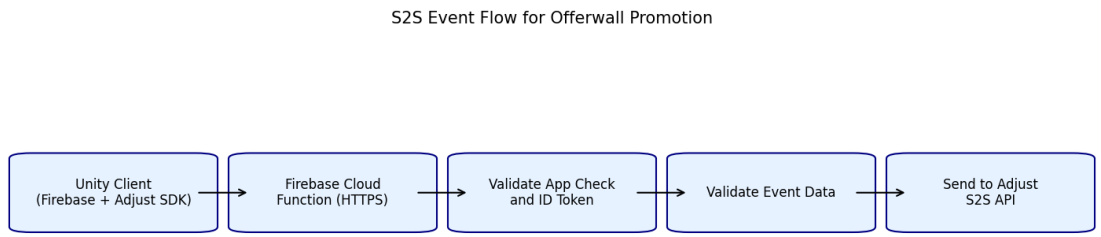
   - *Valid call*: Authenticated user with a fresh ID token; App Check token passes verification; event type is recognised; level number is exactly one greater than the last stored level. The function returns `200 OK` and triggers the Adjust S2S call.

   - *Invalid call*: Missing or expired ID token; App Check header missing; event type unknown; level number lower than or equal to the last stored level. The function returns `401 Unauthorized` or `400 Bad Request` without notifying Adjust.

Backend flow, diagrams and testing strategies

Overall flow

The diagram below shows how the client, server and Adjust services interact during a rewardable event.



Flow diagram showing Unity client verifying App Check and authentication, sending a request to Firebase Cloud Functions, which verifies the tokens, validates the event, calls Adjust S2S API and updates the database.

Process description:

1. Unity game initialises Firebase App Check and Authentication, then registers the Adjust attribution callback.

2. When a user completes a rewardable action, the client checks its Adjust attribution data and confirms that the campaign matches the offerwall promotion.

3. The client obtains an ID token from Firebase Authentication and the App Check token and sends an HTTP POST to the Firebase function with event details and device identifiers.

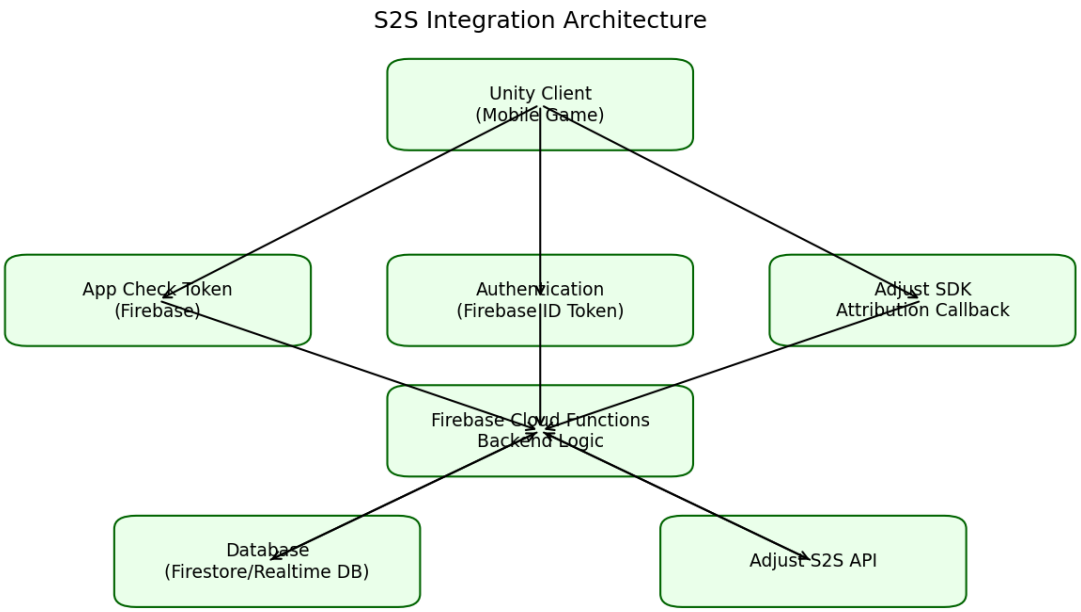
4. The Cloud Function verifies the App Check token and ID token. It queries Firestore to ensure the event is valid.

5. If valid, the server calls Adjust's S2S API with `app_token`, `event_token`, `s2s=1` and the device identifier. The server then updates the user's progress in Firestore and returns success to the client.

6. Adjust records the event and attributes it to the campaign; callback parameters can be returned for further processing.

Architecture diagram

The following architecture diagram highlights the components involved:



Architecture diagram showing mobile app (Unity with Firebase App Check, Firebase Auth, Adjust SDK) connecting to Firebase Cloud Functions (Node.js) which interacts with Firestore/Realtime Database and Adjust S2S API).

Testing strategies

1. **Unit tests** – Write unit tests for your Cloud Function logic using the Firebase Functions Test SDK. Mock the Admin SDK to simulate verifying tokens, retrieving user data and calling Adjust. Test that invalid inputs produce the correct error codes and that valid events update the database as expected.

2. **Local integration testing** – Use `firebase emulators:start` to run the Cloud Function locally. Configure your Unity client or a simple script to send HTTP requests to the emulator. Verify that the function rejects invalid tokens and

validates event parameters. Use the Adjust testing console to confirm that S2S events triggered from the emulator appear correctly.

3. **Adjust test console** – In the Adjust dashboard, use the testing console to monitor incoming S2S events. Fire events from your development environment and confirm that all required parameters are present and that callback parameters are returned.

4. **End-to-end testing** – After deployment, test with a staging version of the app to ensure that App Check and Authentication are enforced. Use a test campaign in Adjust configured for your staging environment. Validate that events are recorded and user rewards are granted only when the user comes from the offerwall campaign.

5. **Monitoring and logging** – Implement comprehensive logging in Cloud Functions (using `functions.logger`) to record incoming requests, validation failures and responses from Adjust. Set up alerts in Google Cloud Monitoring for high error rates or anomalies.